

Синтаксис и семантика Python

Синтаксис и семантика Python

Python Zen:

```
import this
```

```
Beautiful is better than ugly.  
Explicit is better than implicit.  
Simple is better than complex.  
Readability counts.  
There should be one-- and preferably only one --obvious way to do it.  
Now is better than never.
```

(полный текст — философия дизайна языка, 19 принципов)

Отступы

В **Python** для задания блоков потока управления используются отступы — это отличает его от языков со скобками/ключевыми словами.

```
if a == 5:  
    b = 2  
else:  
    b = 7  
    foo()
```

Нельзя смешивать табуляцию и пробелы. Рекомендуется 4 пробела (PEP 8).

Переменные

Имена переменных могут содержать буквы, цифры и `_`, начинаться должны с буквы (`_` в начале имеет семантическую нагрузку). По соглашению имена переменных — с маленькой буквы, имена классов — с большой.

Ключевые слова:

```
and, as, assert, async, await, break, class, continue, def, del,
elif, else, except, finally, for, from, global, if, import, in,
is, lambda, match, nonlocal, not, or, pass, raise, return, try,
while, with, yield
```

(`async` / `await` — с Python 3.5, `match` — «мягкое» ключевое слово с 3.10)

Модель памяти: всё — объект

В Python **абсолютно всё** — объект: числа, строки, функции, классы, модули. У каждого объекта есть:

- **идентичность** (`id(obj)`) — адрес в памяти, не меняется всю жизнь объекта
- **тип** (`type(obj)`) — определяет допустимые операции, не меняется
- **значение** — может быть изменяемым (`list`, `dict`, `set`) или неизменяемым (`int`, `str`, `tuple`)

```
a = 5
type(a), id(a)      # <class 'int'>, 140712834...

def foo(): pass
type(foo)           # <class 'function'> — функция тоже объект!

type(int)           # <class 'type'> — даже класс — объект класса 'type'
```

Небольшие целые (`-5..256`) и короткие строки-идентификаторы **кэшируются и переиспользуются** интерпретатором (интернирование) — это оптимизация реализации, а не гарантия языка:

```
a = 100
b = 100
a is b          # True — маленькие int закэшированы

a = 1000
b = 1000
a is b          # False (в общем случае) — разные объекты с одним значением

a = "hello"
b = "hello"
a is b          # True — интернирование строк-литералов
```

Правило: для сравнения значений - `==`, оператор `is` — только для проверки идентичности (например, `x is None`).

Переменные — это имена-ссылки

В отличие от C/C++, переменная в Python — **не ячейка памяти со значением**, а **имя, привязанное к объекту** (биндинг в пространстве имён, namespace — по сути словарь **имя → объект**).

```
a = [1, 2, 3]
b = a          # b — это ДРУГОЕ ИМЯ для ТОГО ЖЕ объекта
b.append(4)
print(a)       # [1, 2, 3, 4] — a тоже изменился!

a is b         # True — одна и та же идентичность (id совпадает)
```

Присваивание **=** **не копирует значение**, а лишь связывает имя с объектом (rebinding). Это ключевое отличие от языков со статической моделью "переменная = ячейка".

Чтобы получить независимую копию, а не новое имя того же объекта, копирование делается явно:

```
import copy

a = [1, [2, 3]]
b = a           # alias – то же самое имя объекта
c = a.copy()    # неглубокая копия (shallow)
d = copy.deepcopy(a) # глубокая копия (deep)

c[0] = 99
print(a)        # [1, [2, 3]] – не изменился

c[1].append(4)
print(a)        # [1, [2, 3, 4]] – изменился! вложенный список общий
```

`copy()` копирует только верхний уровень; вложенные изменяемые объекты остаются общими между оригиналом и копией. `deepcopy()` рекурсивно копирует всё дерево объектов.

Резолвинг имён: правило LEGB

Когда Python видит имя, он ищет его по цепочке областей видимости **LEGB**:

- **Local** — локальная область (внутри функции)
- **Enclosing** — объемлющая функция (для замыканий)
- **Global** — модуль (глобальная область)
- **Built-in** — встроенные имена (`len`, `print`, ...)

```
x = "global"

def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print(x)      # 'local' — сначала ищем в L
    inner()

outer()
```

Ключевые слова `global` и `nonlocal` явно указывают Python изменять имя не в локальной, а во внешней области.

Изменяемость и подсчёт ссылок (reference counting)

Каждый объект в CPython хранит счётчик ссылок (`ob_refcnt`) — сколько имён/структур на него ссылаются.

```
import sys

a = [1, 2, 3]
sys.getrefcount(a)  # 2 (сама переменная + аргумент getrefcount)

b = a                # refcount += 1
del b                # refcount -= 1
```

Когда счётчик достигает до **0** — память объекта освобождается **немедленно** (в отличие от Java/JS с "ленивым" GC).

Для циклических ссылок (`a` ссылается на `b`, `b` — на `a`) подсчёт ссылок бессилён — их убирает отдельный **циклический сборщик мусора** (`gc` модуль, поколенческий, как доп. уровень поверх refcounting).

Реальный размер объекта в памяти — не то же самое, что "информационный" размер значения:

```
import sys

sys.getsizeof(5)          # 28 байт — у любого int есть заголовок объекта
sys.getsizeof("hello")    # 54 байта
sys.getsizeof([1, 2, 3])  # 88 байт — список хранит указатели, не сами значения
```

Даже пустой объект в CPython занимает десятки байт на служебные поля (`ob_refcnt`, `ob_type`, ...) — отсюда важность специализированных структур (`__slots__`, `array`, `numpy`) при работе с большими объёмами данных.

Изменяемые vs неизменяемые объекты — практические следствия

```
def add_item(lst, item):  
    lst.append(item)    # мутирует объект по ссылке – виден снаружи!  
    return lst  
  
l = [1, 2]  
add_item(l, 3)  
print(l)    # [1, 2, 3]
```

Ловушка — изменяемое значение по умолчанию:

```
def foo(items=[]):    # список создаётся ОДИН РАЗ при определении функции!  
    items.append(1)  
    return items  
  
foo()    # [1]  
foo()    # [1, 1] — неожиданно!
```

Правильно: `def foo(items=None): items = items or []`.

Байт-код и стековая машина CPython

Исходный код Python не выполняется напрямую — он компилируется в **байт-код** (промежуточное представление), который затем исполняет виртуальная машина CPython.

исходный .py → AST → байт-код (.pyc) → интерпретация стековой машиной

Посмотреть байт-код можно через модуль `dis`:

```
import dis

def add(a, b):
    return a + b

dis.dis(add)
```

2	0 RESUME	0
	2 LOAD_FAST	0 (a)
	4 LOAD_FAST	1 (b)
	6 BINARY_OP	0 (+)
	10 RETURN_VALUE	

Как работает стековая машина

CPython — **стековая** (не регистровая) виртуальная машина: инструкции кладут значения на стек операндов и снимают их оттуда.

Для `return a + b` порядок такой:

1. `LOAD_FAST a` — положить значение `a` на вершину стека
2. `LOAD_FAST b` — положить значение `b` на вершину стека
3. `BINARY_OP +` — снять два верхних значения, сложить, положить результат
4. `RETURN_VALUE` — вернуть значение с вершины стека

стек: `[]` → `[a]` → `[a, b]` → `[a+b]` → `[]` (возврат `a+b`)

Каждая функция выполняется в собственном **фрейме** (`frame object`) со своим стеком операндов и локальными переменными — отсюда и трассировка стека вызовов (`traceback`) при исключениях.

Байт-код зависит от версии Python и не переносим между версиями — в отличие от, например, JVM-байткода. Именно поэтому JIT в Python 3.13+ работает поверх этого же байт-кода, дописывая машинный код "по требованию" (copy-and-patch).

Структура объекта в CPython

На уровне C каждый объект Python — это структура `PyObject`, с которой начинается **любой** объект, независимо от типа (зависит от версии CPython):

```
typedef struct _object {  
    Py_ssize_t ob_refcnt;    // счётчик ссылок (см. предыдущие слайды)  
    PyTypeObject *ob_type;   // указатель на объект типа  
} PyObject;
```

Любой `int`, `list`, экземпляр пользовательского класса "сверху" начинается именно с этих двух полей — отсюда минимальные ~16 байт служебных данных у любого объекта (плюс выравнивание).

Объекты переменного размера (`list`, `str`, `tuple`) используют `PyVarObject` — `PyObject` + поле `ob_size` (число элементов).

Объект типа — тоже объект

`ob_type` указывает на **объект типа** (`PyTypeObject`) — он описывает, какие операции поддерживает объект (`__add__`, `__repr__`, ...) и сам является экземпляром `type`:

```
a = 5
a.__class__      # <class 'int'> - то же, что ob_type
type(a) is a.__class__  # True

int.__class__    # <class 'type'>
type.__class__   # <class 'type'> - type - экземпляр самого себя
```

```
a (int) --ob_type--> int (PyTypeObject) --ob_type--> type
```

Именно в объекте типа лежат слоты для методов (`tp_as_number`, `tp_repr`, ...), которые вызываются при `a + b`, `repr(a)` и т. д.

Экземпляр пользовательского класса: `__dict__`

У обычного экземпляра поля хранятся не прямо в C-структуре, а в отдельном словаре атрибутов `__dict__`:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Point(1, 2)
p.__dict__          # {'x': 1, 'y': 2}
p.__dict__['x']     # 1 — то же самое, что p.x

p.z = 3            # добавили атрибут в рантайме
p.__dict__         # {'x': 1, 'y': 2, 'z': 3}
```

Гибкость (можно добавлять атрибуты когда угодно) имеет цену: лишний словарь на каждый экземпляр — накладные расходы по памяти и скорости доступа (хеш-поиск вместо смещения в структуре).

`__slots__` — отказ от `__dict__` ради памяти

Если заранее известен фиксированный набор полей, можно убрать `__dict__` через `__slots__` — CPython выделит под атрибуты компактный массив вместо словаря:

```
class PointSlots:
    __slots__ = ('x', 'y')
    def __init__(self, x, y):
        self.x = x
        self.y = y

p = PointSlots(1, 2)
p.z = 3    # AttributeError: 'PointSlots' object has no attribute 'z'
```

Экономия памяти — обычно **в разы** на объект при массовом создании экземпляров (миллионы точек, узлов графа и т. п.); цена — нельзя динамически добавлять новые атрибуты. Именно так устроены, например, `@dataclass(slots=True)` (с Python 3.10+).

Встроенные типы

bool

- два значения `True` и `False`
- базовые логические операции `or`, `and`, `not`

```
a = True  
b = False  
  
c = a or b  
  
print(c)
```

int

- immutable
- поддерживается длинная арифметика (в Python 2 было два разных типа)

```
a = 2 ** 200 # 1606938044258990275541962092341162602522202993782792835301376
```

- операция целочисленного деления — `//`
- `/` конвертирует значения в числа с плавающей точкой

```
2 // 5 # 0  
2 / 5 # 0.4
```

str

Представление строк:

```
a = "Hello World!"
```

```
a = 'Hello worlds'
```

```
a = """  
    Hello  
    world  
    """
```

Форматирование строк

```
a = 2.1234
c = 5

b = "Formatted numbers: %f, %d" % (a, c)      # старый стиль
b = "Formatted numbers: {:.2f}, {}".format(a, c) # новый стиль
b = f"Formatted numbers: {a:.2f}, {c}"         # f-строки (рекомендуется)

# Python 3.14: t-строки (template strings, PEP 750) – отложенная обработка
t = t"Formatted numbers: {a:.2f}, {c}" # Template-объект, не сразу str
```

Операции со строками

```
v: int = s.find('<str>')      # поиск первого вхождения
v: list = s.split('<str>')    # разделить на строки
v: bool = s.islower()        # s.isupper(), s.isdigit(), s.isspace(), s.isalnum()
v: str = s.replace('<str>', '<str>')
v: str = s.lstrip()           # s.rstrip(), s.strip() – пробелы по краям
v: int = s.count('<str>')     # число вхождений подстроки
v: bool = s.startswith('<str>') # s.endswith('<str>')
v: str = s.join(['<str>', '<str>', '<str>'])
```

Индексирование

```
s = 'Hello World!'
len(s), s[0], s[-1]      # 12, 'H', '!'
b = s[0:3] + s[-2:-1]    # 'Held'
s[::-1], s[1:-1:2]       # '!dlroW olleH', 'el ol'
```

Кодировки

Кодировка символов — однозначное отображение чисел и графических символов, позволяющее хранить и передавать тексты.

Старые однобайтовые кодировки (`ASCII` , `Windows-1251` , `KOI-8R`) поддерживали ограниченный набор символов.

`Unicode` — стандарт кодирования символов большинства письменностей мира. Например, символ Ъ имеет код `U+0463` .

Байтовое представление зависит от конкретной кодировки:

- `UTF-8` — переменная длина (1–4 байта), доминирующая кодировка в вебе
- `UTF-16` — 2 или 4 байта
- `UTF-32` — фиксированно 4 байта

UTF-8

First code point	Last code point	Byte 1	Byte 2	Byte 3	Byte 4
U+0000	U+007F	0xxxxxxx			
U+0080	U+07FF	110xxxxx	10xxxxxx		
U+0800	U+FFFF	1110xxxx	10xxxxxx	10xxxxxx	
U+10000	U+10FFFF	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx

bytes

Неизменяемая последовательность байт, интерфейс схож со строками, но байты не несут семантической нагрузки (не кодируют символы напрямую). Отображаются в кодировке ASCII для удобства.

```
b = b'Hello world'
b = b'Привет мир' # ошибка: bytes can only contain ASCII literal characters
```

```
b = bytes.fromhex('2Ef0 F1f2') # b'.\xf0\xf1\xf2'
```

```
b = "Привет мир".encode("utf-8")
```

```
b = "Привет мир".encode("utf-32")
```

```
b = "Привет мир".encode("windows-1251")
```

Списки

Базовые типы, кодирующие последовательность элементов: `list`, `tuple`, `str`, `bytes`, `bytearray`, `range`. Многие следуют интерфейсу `collections.abc.Sequence`.

Основные операции:

Операция	Действие
<code>x in s</code>	True, если элемент в коллекции
<code>x not in s</code>	True, если элемента нет
<code>s + t</code>	конкатенация
<code>s * n</code>	повтор s n раз
<code>s[i]</code>	i-й элемент
<code>s[i:j]</code>	срез с i до j
<code>s[i:j:k]</code>	срез с шагом k

Список — `list`, изменяемая последовательность элементов, обычно однородных по назначению. Специальный синтаксис:

```
l = [1, 2, 3, None]
```

Основные операции над списками

```
lst.append(5)           # добавление элемента в конец
lst.extend([1, 2, 3])    # добавление коллекции в конец
lst.remove(5)           # удалить первое вхождение
lst.pop()               # удалить последний элемент
lst.count(5)            # число элементов в списке
lst.index(5)            # индекс первого элемента
lst.insert(1, 5)         # вставить el на позицию 1
lst.clear()             # очистить список

lst += [1, 2, 4]         # extend
```

Кортежи

Очень похожи на списки, но неизменяемы. Используются для хранения набора разнородных данных.

```
single = (25,)

person = (25, "Ivan", "Ivanov")

s[0] = 22 # TypeError: 'tuple' object does not support item assignment

age, name, sname = s # "распаковка" элементов кортежа по переменным

t = tuple([1, 2, 3]) # t = (1, 2, 3)
```

Range

Инкапсулирует неизменяемую последовательность целых чисел, используется в циклах.

```
for idx in range(5):  
    print(idx)          # 0, 1, 2, 3, 4  
  
list(range(5))          # [0, 1, 2, 3, 4]  
tuple(range(5))         # (0, 1, 2, 3, 4)
```

Множества

`set` — неупорядоченный набор различных хешируемых объектов. Используется для проверки вхождения, удаления дубликатов и теоретико-множественных операций.

Не поддерживает индексирование и срезы (нет порядка вставки).

В стандартной библиотеке два типа: `set` (изменяемый) и `frozenset` (неизменяемый, хешируемый).

```
s = {1, 2, 3}
s = set([1, 2, 3])
```

```
s = set()
s = set([1, 2, 3])

s.add(5)
s.discard(5)    # удалить элемент (без ошибки, если нет)
s.pop()         # удалить произвольный элемент
s.remove(2)     # удалить элемент (с исключением, если нет)
s.update({1, 2, 3})
s.clear()

r: set = s.difference({1, 2, 3})
r: set = s.union({1, 2, 3})
r: set = s.intersection({1, 2, 3})
r: bool = s.issubset({1, 2, 3})
r: bool = s.issuperset({1, 2, 3})

s |= {1, 2, 3}
s &= {1, 2, 3}
```

```
{1} | {2, 3, 5} - {5}    # {1, 2, 3}
{1, 2} & {2, 5}          # {2}

2 in {1, 2}              # True
3 in {1, 2}              # False
```

Словари

`dict` позволяет однозначно сопоставлять хешируемые объекты с произвольными объектами (ключ → значение). Изменяем.

```
d = {1: 'a', 2: '3', 3: '4'}  
d = dict()  
d = dict([(1, 2), (3, 4)])
```

```
d.items()      # итератор по парам (ключ, значение)  
d.keys()       # итератор ключей  
d.values()     # итератор значений  
d.pop(el)      # удалить ключ, вернуть значение  
d.get(el)      # значение по ключу  
d.get(el, 5)   # значение по ключу или default  
d.update({5: '7', 9: '10'})  
d.clear()
```

Получение значения по ключу

```
s = {1: 'b', 2: 'c'}

s[3] = 'a'
s[2]          # 'c'
s[8]          # KeyError: 8
s.get(8, -1)   # -1
2 in s        # True
s.keys()      # Коллекция ключей
s.values()    # Коллекция значений
```

(с Python 3.7+ словари гарантированно сохраняют порядок вставки)

Итераторы

Итератор — абстракция для прохода по элементам контейнера. Если контейнер/класс поддерживает итерацию, он реализует `__iter__()`, возвращающий объект с `__iter__()` и `__next__()`.

```
l = [1, 2, 3]
it = iter(l)

next(it)    # 1
next(it)    # 2
next(it)    # 3
next(it)    # StopIteration
```

Код

```
lst = [1, 2, 3]

for x in lst:
    action(x)
```

примерно эквивалентен

```
it = iter(lst)
while True:
    try:
        value = next(it)
    except StopIteration:
        break
    action(value)
```

Управляющие конструкции

Цикл `for`

```
s = 0
for i in [1, 2, 3]:
    s += i
```

Итерация по коллекции с индексами:

```
lst = ['a', 'b', 'c']
result = []
for (i, x) in enumerate(lst):
    result.append((x, i))
```

Итерация по индексам:

```
s = 0
l = [1, 2, 3]
for idx in range(len(l)):
    s += l[idx]
```

Цикл `while`

```
i = 0
lst = []
while len(lst) < 6:
    lst.append(i)
    i += 1
```

`break` — досрочно выйти из цикла

`continue` — перейти к следующей итерации

Walrus-оператор (`:=`, Python 3.8+) — присваивание внутри выражения:

```
while (chunk := read_next()) is not None:
    process(chunk)
```

Условный оператор **if**

```
a = 2
if a < 5:
    a = 1
elif a == 5:
    a = 2
else:
    a = 3
```

Структурное сопоставление `match` (Python 3.10+)

Аналог `switch`, но мощнее — с деструктуризацией:

```
def handle(command):  
    match command.split():  
        case ["go", direction] if direction in ("north", "south"):  
            move(direction)  
        case ["go", _]:  
            print("Некуда идти")  
        case ["look"]:  
            look_around()  
        case _:  
            print("Не понимаю команду")
```

Поддерживает сопоставление по типам, структурам классов и последовательностям — полезно для парсеров и обработчиков событий.

List comprehension

Синтаксическая структура для компактного описания операций обработки списков.

```
l = [1, 2, 3]

# список квадратов
l = [x * x for x in l] # [1, 4, 9]

# список квадратов чётных чисел
l = [x * x for x in range(1, 10) if x % 2 == 0]
```

Более сложный пример — список списков:

```
lst = [list(range(x)) for x in range(1, 10) if x % 2 == 1]

# [[0],
#  [0, 1, 2],
#  [0, 1, 2, 3, 4],
#  [0, 1, 2, 3, 4, 5, 6],
#  [0, 1, 2, 3, 4, 5, 6, 7, 8]]
```

Может включать несколько списков:

```
lst = [(x, y) for x in range(1, 20) for y in range(1, 20) if x * y == 16]
# [(1, 16), (2, 8), (4, 4), (8, 2), (16, 1)]
```

Может создавать не только списки, но и словари, множества:

```
d = {x * x: x for x in range(5)}  
# {0: 0, 1: 1, 4: 2, 9: 3, 16: 4}
```

Удобно для работы с текстом:

```
text = ['Hello', 'world', '!']  
text_filtered = [t.lower() for t in text if t.isalpha()]
```

Функции

Функции определяются через `def` или `lambda`.

```
from typing import List, Union

def add(a, b):
    return a + b

div = lambda a, b: a / b

add(2, 3), add('2', '3'), div(4, 2), div(b=2, a=2)
```

Неопределённое число позиционных и именованных аргументов:

```
def foo(a, *args, **kwargs):  
    print(f"{a}")  
    print(f"{args}")  
    print(f"{kwargs}")
```

```
foo(1, 7, 10, c=10)
```

```
# a=1  
# args=(7, 10)  
# kwargs={'c': 10}
```

Функции высшего порядка — можно передавать и возвращать функции:

```
def foo(f, a, b):  
    return f(a, b)  
  
foo(add, 5, 7)  # = 12
```

Функция возвращает функцию (замыкание):

```
def appl(f, a):  
    return lambda x: f(a, x)  
  
add_2 = appl(add, 2)  
add_2(10)  # 12
```

Встроенная сортировка:

```
lst = [3, 2, 5, 1, 7]
sorted(lst)    # [1, 2, 3, 5, 7]

lst.sort()     # in-place
```

С ключом сортировки:

```
lst = [(5, 'the'), (2, 'sun'), (2, 'shines'), (7, 'brightly')]
lst.sort(key=lambda x: -x[0])
# [(7, 'brightly'), (5, 'the'), (2, 'sun'), (2, 'shines')]
```

Итераторы и генераторы

- **итератор** — объект, получаемый из коллекции через `iter()`, следующие элементы — через `next()`
- **итерируемый объект** — объект, из которого можно создать итератор через `iter()`
- **генератор** — частный случай итератора, создаётся специальными функциями с `yield`

```
it = iter([1, 2, 3])  
next(it), next(it), next(it)
```

«Функция-генератор» содержит хотя бы один `yield`. Вызов создаёт итератор, выполнение приостанавливается на `yield`:

```
def gen(val):  
    for i in range(3):  
        yield '%s #%d' % (val, i)  
        print('After yield %d' % i)  
  
for val in gen('From yield'):  
    print(val)
```

```
From yield #0  
After yield 0  
From yield #1  
After yield 1  
From yield #2  
After yield 2
```

Альтернатива — генераторное выражение (круглые скобки вместо квадратных):

```
gen = (x for x in range(1, 100) if x % 2 == 0)
next(gen), next(gen)
# (2, 4)
```

Генераторы экономят память — элементы вычисляются лениво, «на лету», а не хранятся все сразу.

Классы

- в методы явно передаётся ссылка на объект — `self`
- специальные методы («дандеры») начинаются и заканчиваются `--`

```
class Person:
    def __init__(self, name, age, children=None):
        self.name = name
        self.age = age
        self.children = children or []

    def add_child(self, child):
        self.children.append(child)

    def __str__(self):
        return self.name

    def __repr__(self):
        return f'Person("{self.name}", {self.age}, {self.children!r})'
```

Создание экземпляра (оператора `new` в Python нет):

```
person = Person('Ivan Ivanov', 35)
print('{}\n{}\n{}'.format(repr(person), str(person), person))
```

```
Person("Ivan Ivanov", 35, [])
Ivan Ivanov
Ivan Ivanov
```

Экземпляр класса, упрощённо, — словарь полей и методов:

```
vars(person)
# {'name': 'Ivan Ivanov', 'age': 35, 'children': []}
```

Для классов-хранилищ данных удобны `namedtuple` или `@dataclass`:

```
from typing import NamedTuple
from dataclasses import dataclass

@dataclass
class Person:
    name: str
    age: int

class Person2(NamedTuple):
    name: str
    age: int

person = Person(name='Ivanov', age=20)
person, person.name, person.age
```

`@dataclass` — стандарт де-факто для DTO/конфигов; альтернатива — `pydantic` для валидации данных.

Класс может реализовывать протокол итератора:

```
class FibSequence:
    def __init__(self, start0: int, start1: int):
        self.start0 = start0
        self.start1 = start1

    def __iter__(self):
        yield self.start0
        yield self.start1
        while True:
            self.start0, self.start1 = self.start1, self.start0 + self.start1
            yield self.start1
```

```
fib_seq = FibSequence(1, 2)
for i, num in enumerate(fib_seq):
    print(i, num)
    if i >= 5:
        break
```

```
# 0 1 1 2 2 3 3 5 4 8 5 13
```

Асинхронность: `async` / `await`

Python поддерживает кооперативную многозадачность через `asyncio` (с 3.5+, активно развивается по сей день):

```
import asyncio

async def fetch(url: str) -> str:
    await asyncio.sleep(1)  # неблокирующее ожидание
    return f"data from {url}"

async def main():
    results = await asyncio.gather(
        fetch("a.com"), fetch("b.com"), fetch("c.com")
    )
    print(results)

asyncio.run(main())
```

Полезно для I/O-bound задач: сетевые запросы, работа с БД, веб-серверы (`FastAPI`, `aiohttp`).

Современная экосистема инструментов (2025–2026)

- **uv** — менеджер пакетов/окружений/версий Python на Rust, вытесняет `pip` + `venv` + `pyenv`
- **ruff** — линтер и форматтер на Rust, заменяет `flake8` + `black` + `isort`
- **pyright** / **mypy** — статическая проверка типов
- **pytest** — стандарт де-факто для тестирования
- **FastAPI** — современный async веб-фреймворк с автогенерацией OpenAPI
- **pydantic** — валидация данных на основе type hints